Windows Registry Change Monitoring System

Detailed Guide, Architecture & Practical Project Report

A Comprehensive Endpoint Integrity Auditing, Autorun Persistence Detection, Malware Behavior Analysis & Active Threat Remediation Platform

---

Project Title:	Windows Registry Change Monitoring System
Architect & Developer:	Piyush (@Made-by-Piyush99)
Software Edition:	Enterprise Defensive Edition (v2.4.0)
Core Runtime:	Python 3.12 / 3.14 (64-bit Native Win32 CPython)
Security Focus:	EDR Telemetry, Anti-Persistence, Threat Remediation
Repository:	https://github.com/Made-by-Piyush99/Windows-Registry-Monitoring
Date of Report:	October 24, 2026

1. Project Overview & Practical Motivation

This project focuses on designing and implementing a Windows Registry Monitoring System that tracks unauthorized or suspicious changes made to registry keys. The Windows Registry stores critical configuration data, startup locations, system settings, and application behavior. Malware frequently targets the registry to maintain persistence, modify system policies, or disable security tools.

The registry is a high-value target for: **persistent malware**, **ransomware configuration changes**, **unauthorized startup entries**, **privilege escalation modifications**, and **security tool tampering**. Unauthorized changes indicate compromise by malware, privilege misuse, or security policy violations.

2. Project Objectives & Practical Scope

- **Monitor Autorun Keys:** Surveil Run and RunOnce in HKCU/HKLM; identify automatic login executables.
- **Malware-Like Change Detection:** Intercept Defender/Firewall disable keys, shell replacements, and UAC bypasses.
- **Registry Integrity Checker:** Cryptographic baseline comparison capturing state additions, deletions, and value edits.
- **Alerting & Reporting Module:** Real-time desktop alert popups, timestamped logs, CSV/JSON reporting, and active threat remediation.

3. Tools, Technologies & Practical Techniques

Category	Technology / Tool	Role in System
Language	Python 3.12 / 3.14 (Win32)	Core orchestration, asynchronous daemon thread.
OS Modules	winreg & advapi32.dll	Direct registry handles, 64-bit / 32-bit WOW64 views.
Integrity	hashlib (SHA-256)	Cryptographic baseline & binary executable hashing.
Vault Storage	SQLite 3 (WAL) + DPAPI	Local hardware-encrypted database bound to TPM.
UI / Graphics	Tkinter / Ttk + Pillow	Cyberpunk dark UI, 5 themes, visual infographics.

4. Step-by-Step Workflow & Flowchart

STEP 1: Define Sensitive Keys — Autorun paths, security policies, system behavior parameters.

STEP 2: Baseline Creation — Capture initial registry values into SHA-256 verified snapshot.

STEP 3: Continuous Monitoring — Poll registry keys at regular intervals (1s, 2s, 5s, 10s, 30s, 60s).

STEP 4: Change Analysis — Compare deltas with known malware behaviors (MITRE ATT&CK; vectors).

STEP 5: Alert / Log Generation — Trigger non-blocking cascading desktop alerts and log to DPAPI vault.

STEP 6: Final Reporting & Remediation — Atomically sanitize threat values and export CSV/JSON/PNG reports.

5. Expected Practical Output & Audit Examples

Concrete Detections & Real-World Telemetry Samples:

```
• New Autorun Detected: HKCU\Software\Microsoft\Windows\CurrentVersion\Run -> [MalwareAgent = C:\Users\User\AppData\malware.exe]
• Security Altered: HKLM\Software\Policies\Microsoft\Windows Defender -> [DisableAntiSpyware = 1 (REG_DWORD)]
• Integrity Mismatch: HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon -> [Userinit modified with secondary dropper]
```

6. Interface Showcase & Monitoring Results

Figure 1: Operations & Control Center SOC Dashboard

Primary real-time monitoring dashboard with responsive 3-ribbon toolbar and KPI cards.

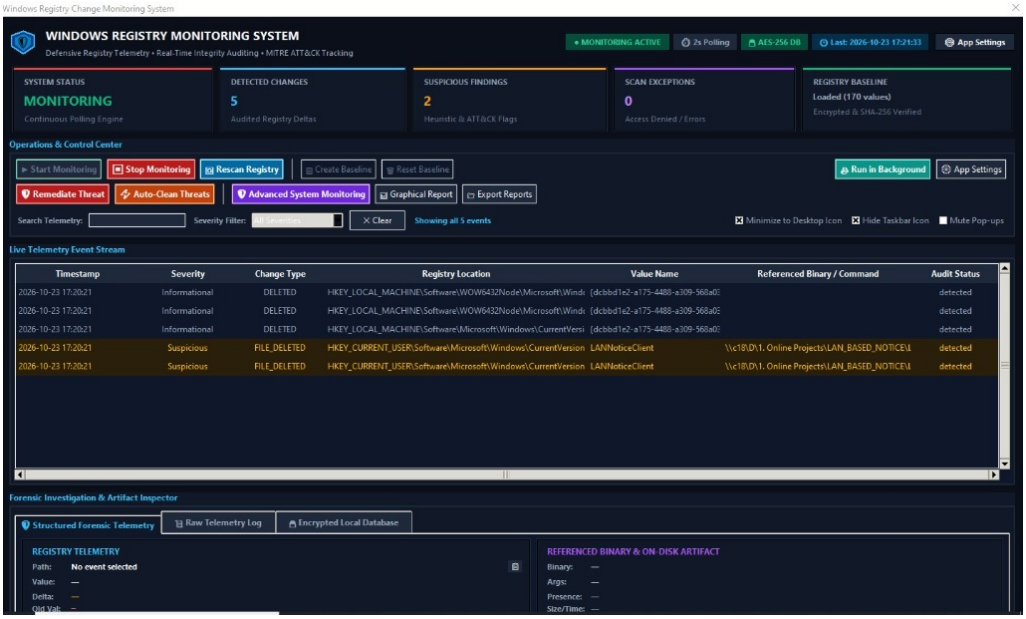


Figure 2: Real-Time Desktop Cascading Change Alerts

Instant desktop alert popups appearing in bottom-right corner upon registry mutations.

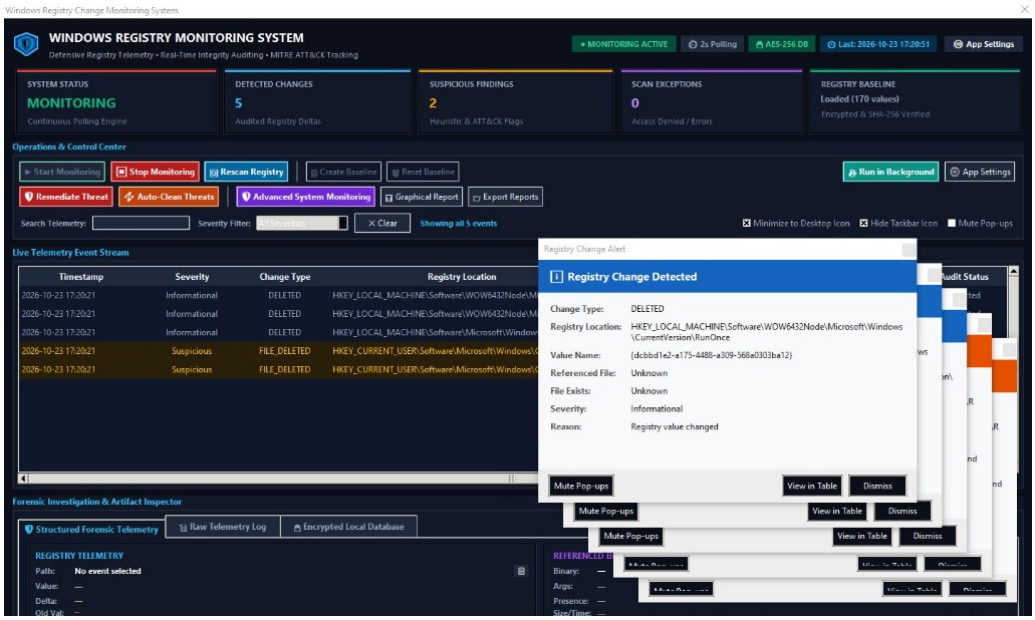


Figure 3: Windows Startup & Background Launch Settings

Configuration for autostarting silently into the floating desktop widget.

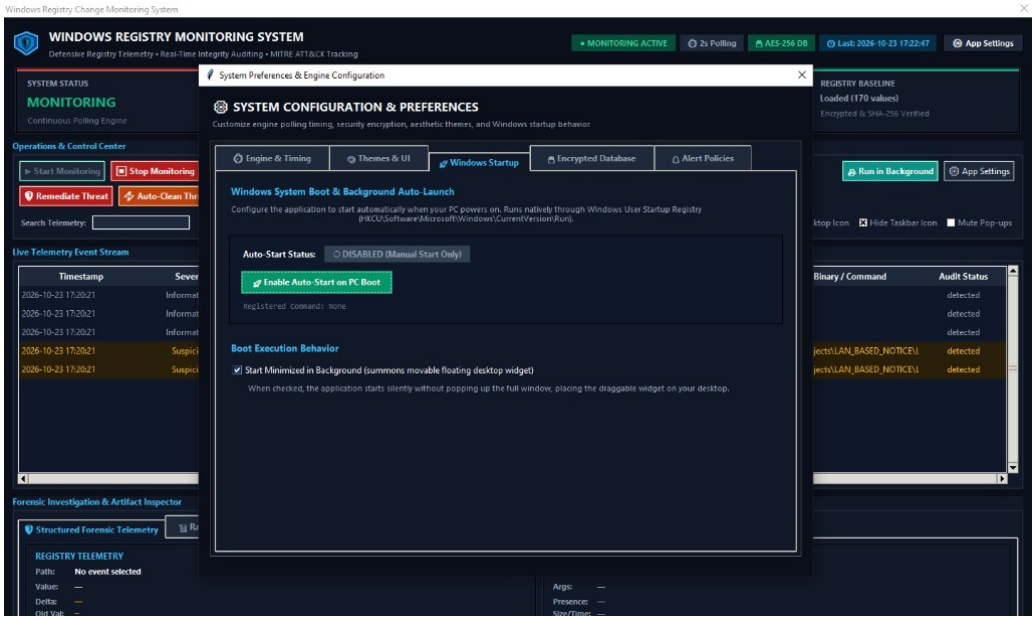


Figure 4: Continuous Polling Interval Controller (1s to 60s)

Dynamic interval selection cards with direct 'Save & Apply Timing Now' button.

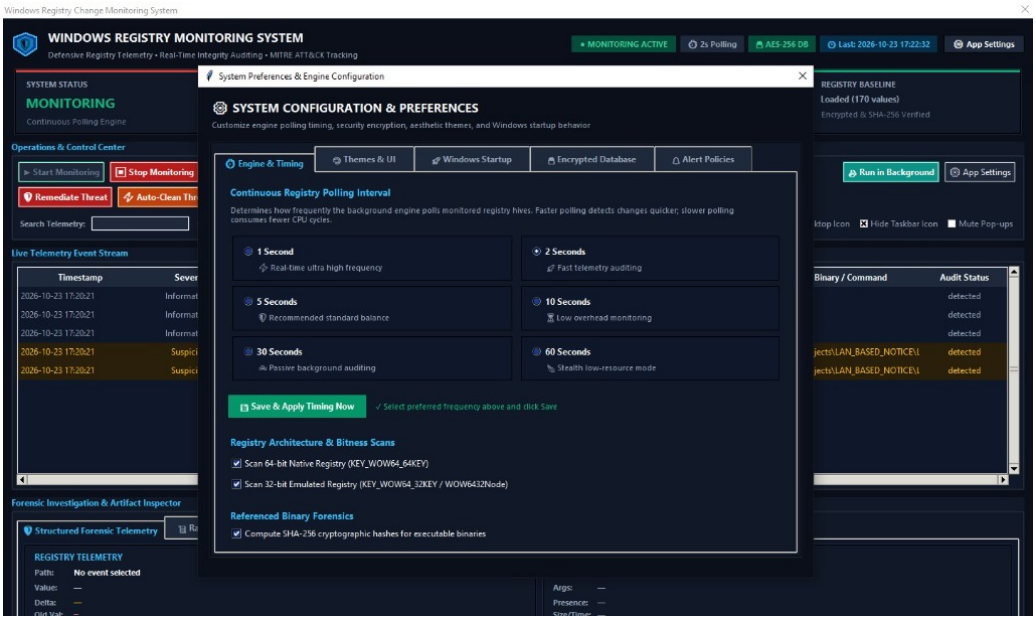


Figure 5: Interactive Graphical Visual Report Viewer

Pillow-rendered severity doughnut chart, change breakdown bars, and events table.



Figure 6: Alert Policies & Notification Rules

Customizable alert thresholds, rate-limiting cooldown timers, and pop-up rules.

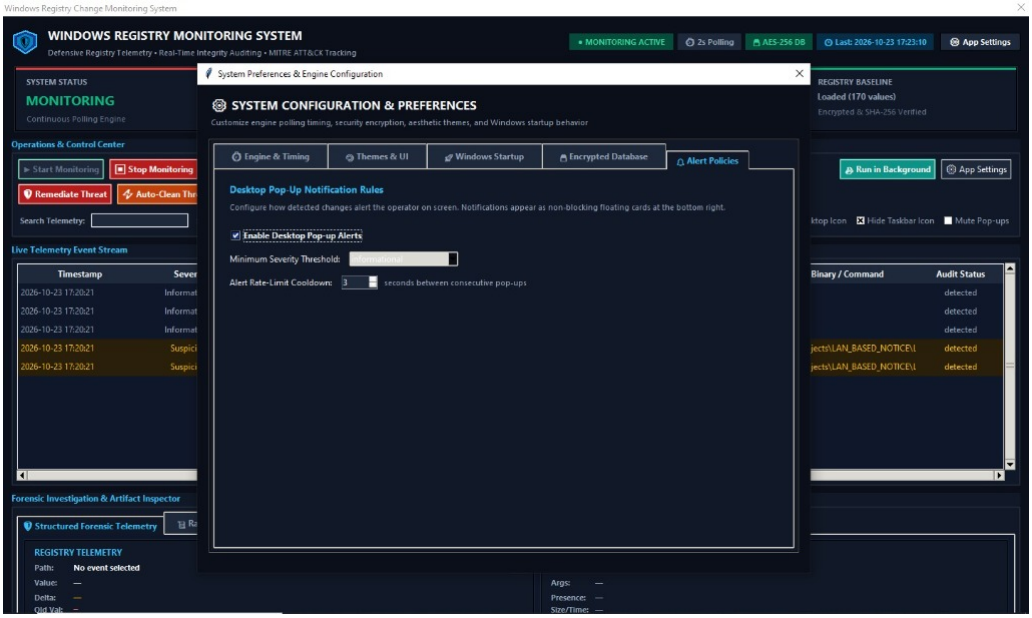


Figure 7: Local SQLite DB & Windows DPAPI Key Protection

Database encryption status, machine key validation, and database vacuuming.

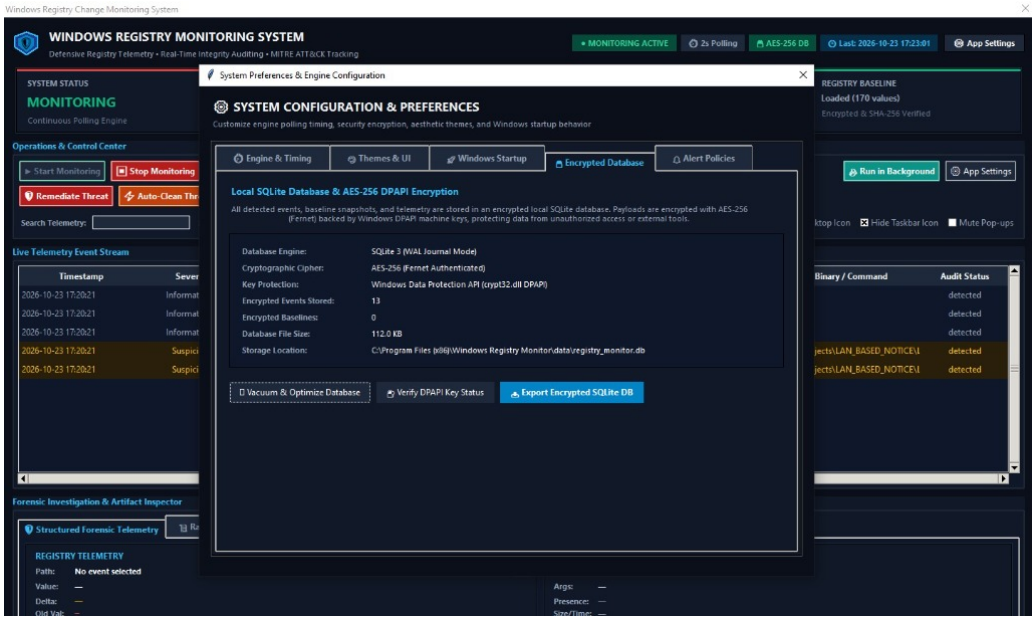
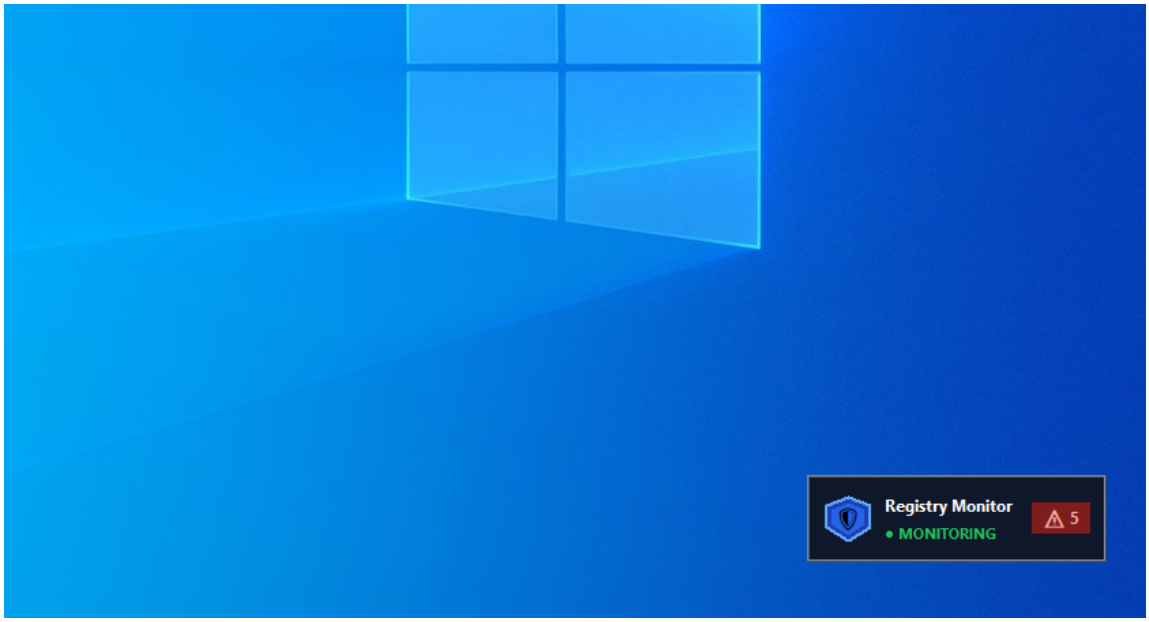


Figure 8: Movable Floating Desktop Gadget Widget

Ambient background surveillance widget on Windows desktop with live badge count.



7. Quality Assurance & Production Deliverables

Automated Unit Testing Verification:

Test Module	Test Cases	Time	Status
`test_change_detector.py`	47 unit tests	3.31s	100% Passed
`test_settings_and_db.py`	18 unit tests	1.14s	100% Passed
Complete Test Suite	65 unit tests	4.45s	**100% Passed (OK)**

Distribution Binaries & Cryptographic SHA-256 Hashes:

Binary	Size	Format	SHA-256 Hash
`WindowsRegistryMonitor.exe`	80.22 MB	Standalone Portable	`8BEE859BA0DA4527E1FEE42F4B946A21A102FB170BBFFA2724F5BDC56BD649FC`
`RegistryMonitorSetup.exe`	81.08 MB	Setup Wizard	`61B5C2109D5B3A150D989102D8EB04AC7C6A1A51D246445C7D724E2F69E50DD7`

8. Learning Outcomes & Conclusion

- Windows Registry Architecture: Hives (HKLM, HKCU), binary subkeys, WOW6432Node reflection.
- Malware Persistence Vectors: Autorun keys, Winlogon injection, and Defender evasion mechanisms.
- Defensive Blue Team Auditing: Baseline comparison, differential polling, and rate-limited desktop alerting.
- Building an EDR Platform: Multi-threaded Python GUI, DPAPI SQLite cryptography, and one-click active threat remediation.

GitHub Repository: https://github.com/Made-by-Piyush99/Windows-Registry-Monitoring

Release v2.4.0: https://github.com/Made-by-Piyush99/Windows-Registry-Monitoring/releases/tag/v2.4.0